

Text as Data

W4: Dictionaries and Sentiment Analysis

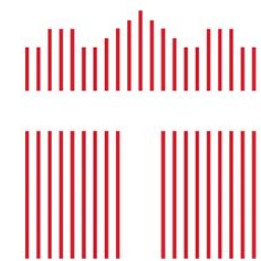
Yen-Chieh Liao 廖彥傑

2026-10-02

Fall 2026 (115-1) · National Taiwan University (GIND)

NTLDEV 7191 · Fri. 16:30–18:20 · 國發 204

Last compiled: 2026-10-02 09:25:52 (Asia/Taipei)



**National
Taiwan
University**
國立臺灣大學

Learning objectives

- **Measurement**

Define what a dictionary score is intended to measure.

- **Dictionary construction**

Build and revise a word or phrase list using political texts.

- **Scoring**

Turn dictionary matches into scores; explain the denominator and texts with no matches.

- **Validation**

Use published evidence and human judgments to assess validity.

- **Own project:**

Plan a dictionary measure for your research.

A political research question

When do MPs describe household economic conditions negatively?

- *“Families cannot afford rent or energy bills.”*
- Possible comparison: speakers, periods, or government status.
- Read both the household issue and the evaluation: `rent` identifies the topic; `cannot afford` describes hardship.

Mentioning inflation does not tell us how an MP evaluates household conditions.

Attention, sentiment, and stance

Concept	Question	Example
Issue attention	What is mentioned?	This speech discusses energy bills.
Sentiment	How is something evaluated?	High energy bills are hurting families.
Stance	What position is taken toward a target?	I support the government's price cap.

Negative language about a problem can accompany support for a policy.

A measurement workflow

1. **Find relevant sentences:** household income, living costs, and affordability.
2. **Code the evaluation and its target:** how are household conditions described?
3. **Validate against human judgments:** include dictionary matches and nonmatches.
4. **Compare speakers or periods:** relevant sentences judged negative / all relevant sentences.

Today we practise two parts separately: counting issue mentions and scoring positive/negative words. Together, they do not yet identify evaluations of household conditions.

Construct, indicator, and claim

Example: “Families cannot afford rent or energy bills.”

Level	Concrete example
Construct	How much an MP talks about household costs
Indicator	Count matches such as <code>rent</code> and <code>energy bills</code> (one phrase match)
Score	Speech A: 4 matches / 100 words = 4 per 100 ; speech B: 4 / 200 = 2 per 100
Claim	Speech A has twice as many household-cost matches per 100 words as speech B

This counts mentions. To measure negative evaluations, we also need to identify language such as “cannot afford.”

One corpus, different units of analysis

Text unit	Political question
Sentence	Which statements refer to household hardship?
Speech	How much of a contribution concerns hardship?
Speaker-period	How does a speaker’s attention change over time?

Our lab sample is **5,000 UK House of Commons contributions, 1945–2025**.

Keep speaker, date and source IDs when splitting or combining texts.

Set up once for today's examples

Each document is one parliamentary speech contribution. Run C01 once in R to load the data and examples; keep this R session open.

```
# C01: Setup
# First use: uncomment the next line to install.
# install.packages(c("quanteda", "quanteda.textstats"))
library(quanteda)
library(quanteda.textstats)
paths <- c("workflow.R", "W4/classroom/workflow.R",
           "classroom/workflow.R", "../classroom/workflow.R")
found <- paths[file.exists(paths)]
if (length(found) == 0) found <- file.choose()
setwd(dirname(normalizePath(found[1])))
source("workflow.R")
```

Bag of words: feature counts (1/2)

Predict: How many times will `prices` appear in row A?

```
# C02: Counts and document length
library(quantda)
demo <- c(A = "Prices rise. Prices worry voters.",
          B = "Prices fall.")
demo_tokens <- tokens(demo, remove_punct = TRUE)
demo_dfm <- dfm(demo_tokens)
selected <- dfm_match(demo_dfm, c("prices", "rise", "fall"))
as.matrix(selected)
ntoken(demo_tokens)
```

Compare: Why do the selected row totals differ from document length?

Bag of words: feature counts (2/2)

- **Rows = documents; columns = selected words.** Each cell counts occurrences.
- **Read row A:** `prices` appears twice, `rise` once, and `fall` zero times.
- **Length is different:** A has 5 word tokens, but its displayed counts sum to 3. `worry` and `voters` are not shown.

Code: `as.matrix(selected)`

```
      features
docs prices rise fall
  A      2    1    0
  B      1    0    1
```

Code: `ntoken(demo_tokens)`

```
A B
5 2
```

What word counts leave out

The government does not support the opposition.

The opposition does not support the government.

Same bag of words; different actor and target.

Frequency: occurrences and speeches (1/2)

Predict: Is a frequent word also used in many different speeches?

```
# C03: Frequency in real speeches (run C01 first)
recap_dfm <- dfm(discovery_tokens)
recap_dfm <- dfm_remove(recap_dfm, stopwords("en"))
# Keep words made of English letters only.
recap_dfm <- dfm_select(recap_dfm, "^[a-z]+$", valuetype = "regex")
freq <- textstat_frequency(recap_dfm)
head(freq, 6)
```

`frequency` counts occurrences; `docfreq` counts speeches with a match.

Try: Change 6 to 12. Which words describe procedure rather than policy?

Frequency: occurrences and speeches (2/2)

Code: `head(freq, 6)`

	feature	frequency	rank	docfreq	group
1	hon	6514	1	2826	all
2	government	3190	2	1320	all
3	right	2762	3	1470	all
4	member	2395	4	1129	all
5	can	2391	5	1221	all
6	people	2273	6	995	all

- `frequency`: total occurrences. `hon` appears **6,514 times**.
- `docfreq`: speeches containing the word. `hon` appears in **2,826 speeches**.
- `rank`: order by frequency. `group = all`: speeches are pooled together.

KWIC: reading words in context (1/2)

KWIC = keyword in context. Will `security` always refer to defence?

```
# C04: Context in real speeches (run C01 first)
live_contexts <- kwic(discovery_tokens, "security", window = 5)
context_table <- as.data.frame(live_contexts)
columns <- c("docname", "pre", "keyword", "post")
head(context_table[, columns], 2)
```

In pairs: Classify two matches. Then change the window to 12.

KWIC: reading words in context (2/2)

Code: `head(context_table[, columns], 2)`

```
      docname      pre keyword      post
1 speech_0024 as a basis of international security , is essential to the
2 speech_0246  raise the question in the security council . we advanced £
```

- **Each row is one match.** `docname` identifies the speech.
- **Read across:** `pre` → `keyword` → `post`; up to five tokens on each side.
- **Read the phrase:** row 1 shows `international security`; row 2 shows `security council`. Use the surrounding words to interpret the match.

Collocations: candidate phrases (1/2)

Collocations: adjacent words with a strong statistical association.

```
# C05: Phrases in real speeches (run C01 first)
coll <- textstat_collocations(discovery_tokens, size = 2, min_count = 20)
row_order <- order(coll$lambda, decreasing = TRUE)
coll <- coll[row_order, ]
head(coll, 6)
first_phrase <- phrase(coll$collocation[1])
phrase_context <- kwic(discovery_tokens, first_phrase, window = 5)
head(phrase_context, 2)
```

Try: Raise `min_count` to 50. Are the leading phrases about household costs?

Collocations: candidate phrases (2/2)

Code: `head (phrase_context, 2)`

Keyword-in-context with 2 matches.

```
[speech_1885, 29:30] the british-owned tea estates in | sri lanka | . there has been considerable  
[speech_1885, 80:81] to extend its stay in | sri lanka | to investigate the conditions of
```

- **The matched phrase is `sri lanka`.** It ranks first after we sort the candidate phrases by `lambda`.
- **Read each line:** speech ID and token positions → words before → matched phrase → words after. Both matches come from the same speech.
- **A strong collocation is only a candidate.** This place name does not, by itself, measure household costs.

Keyness: comparing groups (1/2)

Keyness: which words are distinctive of Conservative versus Labour speeches?

```
# C06: Relative word use (run C01 and C03 first)
party_dfm <- dfm_subset(recap_dfm,
  party %in% c("Conservative", "Labour"))
party_dfm <- dfm_group(party_dfm, groups = party)
party_dfm <- dfm_trim(party_dfm, min_termfreq = 1)
keys_con <- textstat_keyness(party_dfm, target = "Conservative")
keys_lab <- textstat_keyness(party_dfm, target = "Labour")
head(keys_con, 3)
head(keys_lab, 3)
```

Compare: Each run uses a different target. Which words stand out for each party?

Keyness: comparing groups (2/2)

Target: Conservative				Target: Labour			
Word	chi2	Con.	Lab.	Word	chi2	Con.	Lab.
aircraft	44.37	116	31	waste	61.43	31	124
cyberstalking	39.62	42	0	tory	49.50	11	73
stalking	33.96	36	0	advertising	38.72	12	64

- **Two targets:** each list shows the three highest positive scores for the party named above.
- **Compare relative use:** keyness accounts for each group’s token total; the counts alone do not determine it.
- **Interpretation:** these words are distinctive in this sample, not necessarily the most frequent or positive in sentiment.

Actual C06 results: chi2 rounded to two decimals; p omitted. Con. = Conservative; Lab. = Labour. Counts are word occurrences.

What is a dictionary?

A dictionary maps **expressions** to **predefined categories**.

```
library(quanteda)
dic <- dictionary(list(
  household_costs = c("inflation", "rent", "energy bills"),
  employment = c("unemployment", "job security")
))
```

`dic` stores the dictionary for the next example.

Mentioning household costs does not establish economic hardship.

Applying our dictionary

```
# Run the preceding slide first to create dic.
texts <- c("Rent and energy bills are rising.",
          "We need job security.")
example_tokens <- tokens(texts, remove_punct = FALSE)
matched <- tokens_lookup(example_tokens, dic, capkeys = FALSE)
counts <- dfm(matched)
as.matrix(counts)
```

Document	household_costs	employment
text1	2	0
text2	0	1

rent + energy bills give **2 matches**; job security gives **1**.

Existing, adapted, or new?

Choice	Main question
Existing dictionary	Does its original domain fit this corpus?
Adapted dictionary	Which changes improve performance here?
New dictionary	Can we document coverage and exclusions?
Another method	Is the category too context-dependent for word rules?

Availability is not evidence of suitability.

Next: Whether adapting or building, decide which expressions belong in the dictionary.

Building or adapting a word list

1. Write down what should count, with one example that should not.
2. Read passages that do and do not discuss the issue.
3. List words and phrases that signal the issue.
4. Check false matches and relevant passages the list misses.

For household costs: distinguish **energy bills** from a **parliamentary bill**.

POS tagging can help us find candidate words, which still need review in context.

POS tagging with UDPipe (1/3)

Optional extension: POS → candidate terms → human review.

```
# 001: POS candidates (model download on first run)
# install.packages("udpipe") # run once before class
library(udpipe)
dir.create("udpipe_models", showWarnings = FALSE)
info <- udpipes_download_model("english-ewt",
  model_dir = "udpipe_models", overwrite = FALSE)
model <- udpipes_load_model(info$file_model)
text <- "Housing costs are unaffordable."
annotation <- udpipes_annotate(model, x = text, parser = "none")
ann <- as.data.frame(annotation)
ann[, c("token", "lemma", "upos")]
```

Discuss: Which tokens would you keep as dictionary candidates?

POS tagging with UDPipe (2/3)

Code	What it does
<code>library(udpipe)</code>	Loads the installed R package.
<code>dir.create(...)</code>	Creates a folder for the model file.
<code>udpipe_download_model</code>	Downloads model data; keeps an existing file.
<code>udpipe_load_model</code>	Reads that model into memory.
<code>udpipe_annotate</code>	Applies the model to our sentence.
<code>as.data.frame(annotation)</code>	Converts the result into a table.

`ann[, c("token", "lemma", "upos")]` keeps **all rows** and selects **three columns**.

POS tagging with UDPipe (3/3)

token	lemma	upos
Housing	housing	ADJ
costs	cost	NOUN
are	be	AUX
unaffordable	unaffordable	ADJ
.	.	PUNCT

token: observed form · **lemma:** predicted base form · **upos:** predicted word class

Discuss: Would you accept `Housing` → `ADJ`? Which words are useful dictionary candidates?

What can POS help us find?

Approach	Role of POS and token relations
Candidate filtering	Keep nouns or adjectives for human review.
TextRank	Filter candidates with POS; rank words in a co-occurrence network.
Dependency-based extraction	Follow grammatical links between words, such as a noun and its modifier.

Co-occurrence: words appear near each other. **Dependency:** a parser predicts a grammatical link.

Our O01 used `parser = "none"`; dependency extraction needs parsing enabled.

TextRank: ranking candidate words (1/2)

Obama's 2013 inaugural address, included in `quanteda`. Run `O01` first to load `model`.

```
library(quanteda)
# install.packages("textrank") # run once
library(textrank)
speech <- corpus_subset(data_corpus_inaugural, Year == 2013)
text <- as.character(speech)
ann <- as.data.frame(udpipe_annotate(model, x = text,
                                     parser = "none"))
keep <- ann$upos %in% c("NOUN", "ADJ")
result <- textrank_keywords(ann$lemma, relevant = keep,
                           ngram_max = 2, sep = " ")
scores <- sort(result$pagerank$vector, decreasing = TRUE)
round(head(scores, 6), 4)
```

TextRank: ranking candidate words (2/2)

Actual output: Obama 2013, English EWT model

```
new    same endure nation common    hard
0.0161 0.0096 0.0095 0.0094 0.0092 0.0083
```

- **Nodes:** candidate lemmas. **Edges:** adjacent noun/adjective pairs.
- `new` links to `job` twice; to `response`, `challenge`, `idea` and `industries` once each.
- PageRank uses weighted connections and neighbours' scores; raw frequency alone does not determine rank.
- **Review the result:** `new` is broad; some forms of `endure` were tagged as nouns. Neither is automatically a useful dictionary entry.

```
pairs <- cooccurrence(ann$lemma, relevant = keep)
head(pairs, 3) # inspect the links used to build the network
```

Named entities with Flair (1/2)

Optional Python demo. POS labels word classes; NER identifies names and their types.

```
# Install once in a terminal: python -m pip install flair
from flair.data import Sentence
from flair.models import SequenceTagger

tagger = SequenceTagger.load("ner-fast")
sentence = Sentence(
    "Barack Obama spoke at the United Nations in New York.")
tagger.predict(sentence)
for entity in sentence.get_spans("ner"):
    print(entity.text, entity.get_label("ner").value)
```

`load()` loads the model; `predict()` finds entities; `get_spans()` retrieves them.

Named entities with Flair (2/2)

“Barack Obama spoke at the United Nations in New York.”

```
Barack Obama PER
United Nations ORG
New York LOC
```

Detected name	Label	Interpretation
Barack Obama	PER	Person
United Nations	ORG	Organization
New York	LOC	Location

- NER groups words into a name: **United Nations** is one entity span.
- Use these spans to find mentions, then inspect their context.
- An organization mention does not tell us whether the speaker supports it.

Entity mention is not stance

The minister praised the NHS but criticized its funding.

Keep discovery separate from evaluation

Whether terms come from reading, POS or TextRank, the next step is validation.

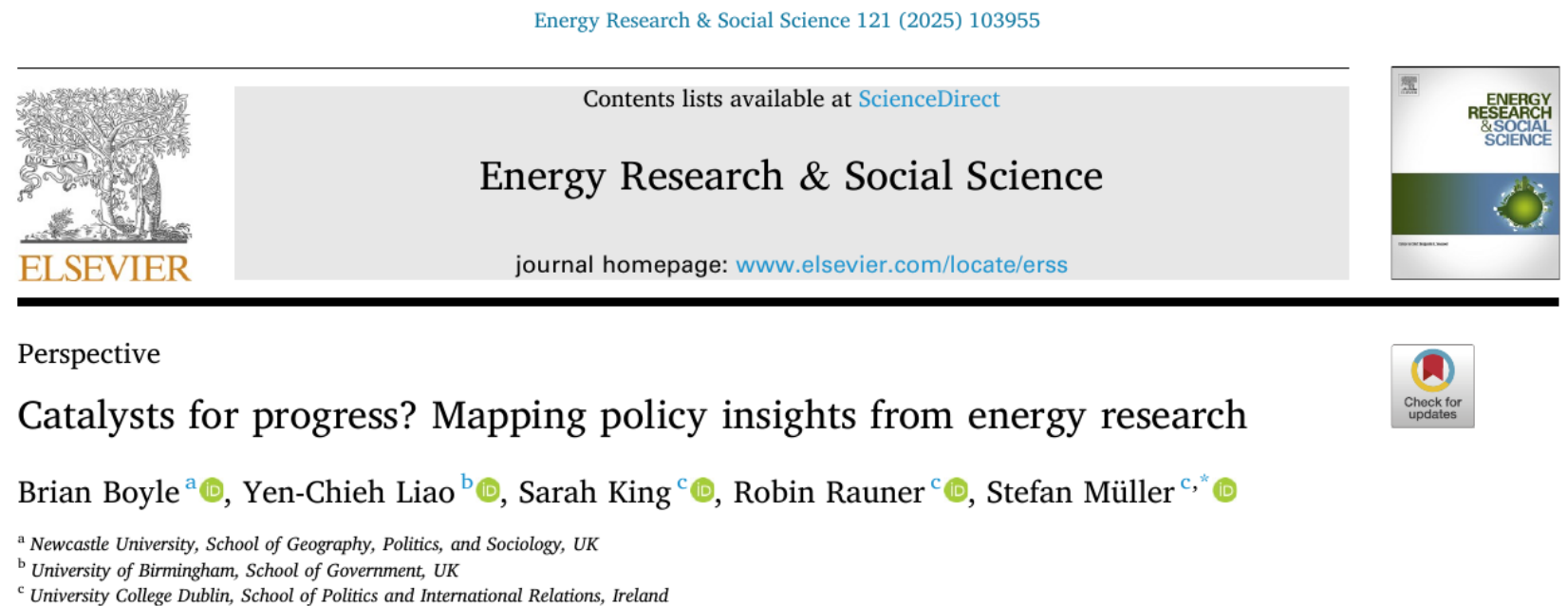
Discovery texts → candidate terms → human review → frozen dictionary

Held-out texts → independent human labels → error analysis

- Record the dictionary version and inclusion rules.
- Do not keep tuning on the final evaluation sample.
- Recheck performance when the language or period changes.

Case: energy research and policy relevance

- Which abstracts communicate **policy relevance**?
- **270,537 abstracts**, 100 journals, 2010-2023.
- A custom dictionary identifies relevant statements.
- Expressed relevance is not policy uptake.



From abstracts to a policy-relevance measure

- **Define:** presence or absence of a policy-relevant statement.
- **Develop:** 400 human-coded abstracts; seed terms and qualitative review.
- **Refine:** retain added terms that improve positive-class F1.
- **Evaluate:** 200 separate abstracts, not used for tuning.

Reported F1: **0.90 development / 0.86 held out.**

F1 combines precision and recall; we will calculate both later.

Initial approach: We first tried fine-tuning transformer models, but the results were not satisfactory.

Policy relevance varies across journal groups

About 15% of abstracts were classified as policy relevant.

Only 8% explicitly mentioned `policy` or `policies`.

Journal aims and scope	Policy-relevant abstracts
Explicitly refer to policy	26.7%
Do not explicitly refer to policy	3.7%

These are descriptive text patterns, not evidence of policy uptake.

A small example we can check by hand

Text	Broad <code>bill</code> *	<code>energy bills</code>
Energy bills are rising.	1	1
This bill reforms parliament.	1	0
Families cannot afford heating.	0	0

A phrase removes one false positive but still misses a relevant passage.

Matching with quanteda

Predict: Which sentence will the phrase dictionary match?

```
# C07: A complete phrase-lookup example
library(quanteda)
examples <- c("Energy bills are rising.",
              "This bill reforms parliament.",
              "Families cannot afford heating.")
example_tokens <- tokens(examples, remove_punct = FALSE)
cost_dict <- dictionary(list(costs = "energy bills"))
matched <- tokens_lookup(example_tokens, cost_dict)
cost_counts <- dfm(matched)
as.matrix(cost_counts)
```

Change: Add "cannot afford heating". Which error changes?

Exact terms and wildcard rules (1/2)

Predict: What will `bill*` retrieve besides bills?

```
# C08: Wildcards in real speeches (run C01 first)
wild <- kwic(discovery_tokens, "bill*", window = 0)
exact <- kwic(discovery_tokens, c("bill", "bills"),
              valuetype = "fixed", window = 0)
c(wildcard = nrow(wild), exact = nrow(exact))
word_counts <- table(wild$keyword)
word_counts <- sort(word_counts, decreasing = TRUE)
head(word_counts, 8)
```

Discuss: Does an exact match solve the substantive ambiguity?

Exact terms and wildcard rules (2/2)

Actual C08 output

wildcard	exact
1774	1539

- `bill*`: matches any suffix after `bill`, including `billion`.
- Exact `bill` / `bills`: $1,470 + 69 = 1,539$ matches.
- Exact matching still mixes legislation and household bills.

These are match counts, not numbers of speeches. Use KWIC to check meaning.

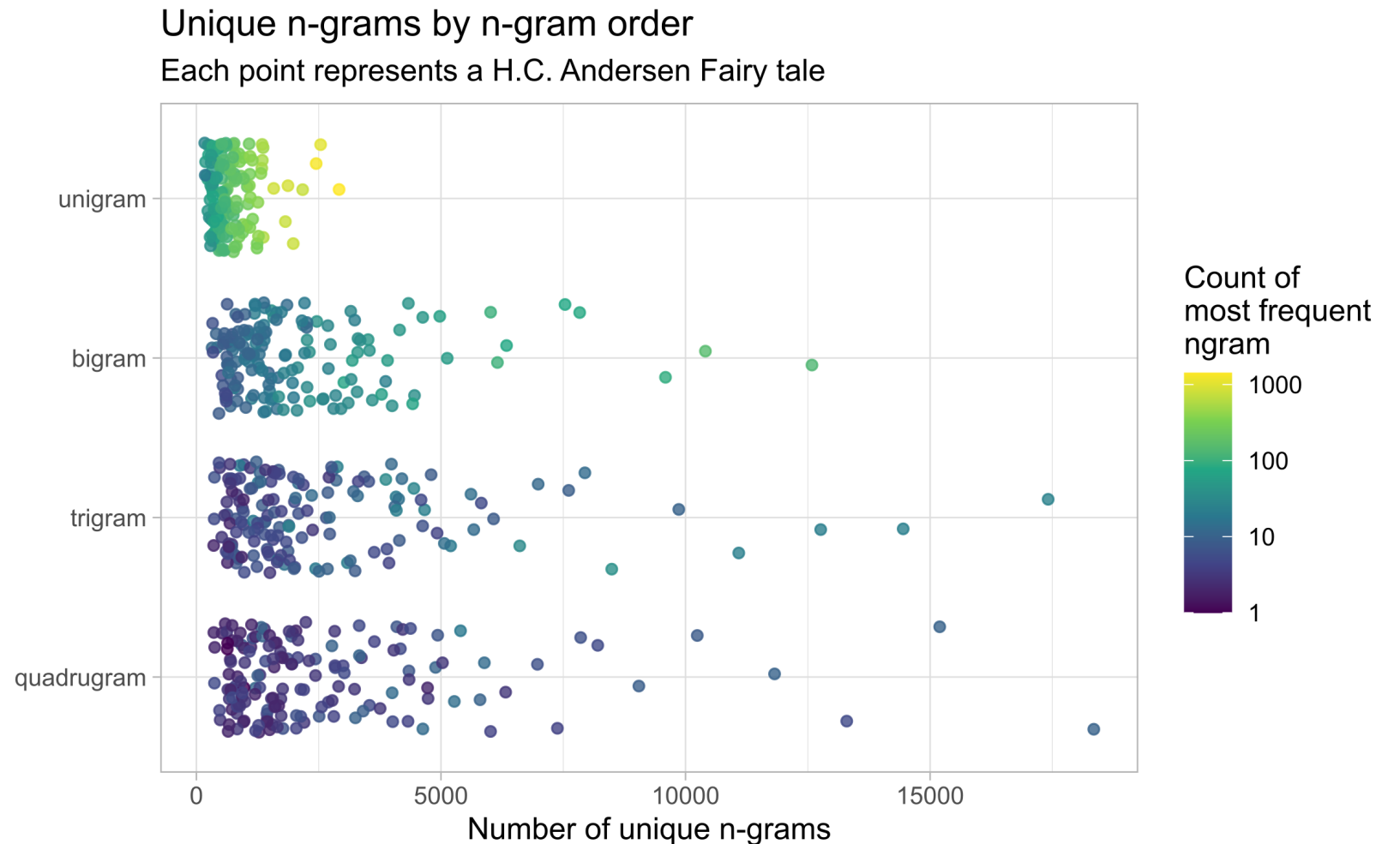
The eight most frequent wildcard matches

Token	Count	Token	Count
bill	1,470	billions	18
billion	157	billericay	5
bills	69	bill’s	3
bill’s	24	billâ	3

Different apostrophes and an apparent encoding artifact remain visible in the output.

Phrase rules preserve useful context

- Phrases distinguish **energy bills** from legislative bills.
- Longer n-grams capture more local context.
- Specific expressions can also be **rare**.
- Match phrases before removing stopwords.



Note: Each dot = one Andersen story at a given n-gram length. X = number of distinct n-grams; rows = 1–4-word sequences. Colour = occurrences of the most frequent n-gram (log scale: yellow = more; purple = fewer).

Document length and the denominator (1/2)

Predict: Will counts and rates identify the same top speeches?

```
# C09: Rankings on real speeches (run C01 first)
discovery_ids <- docnames(discovery_tokens)
rates <- subset(comparison, doc_id %in% discovery_ids)
rates <- subset(rates, tokens >= 50)
cols <- c("doc_id", "broad", "tokens", "broad_per100")
raw_order <- order(rates$broad, decreasing = TRUE)
rate_order <- order(rates$broad_per100, decreasing = TRUE)
head(rates[raw_order, cols], 3)
head(rates[rate_order, cols], 3)
```

Compare: What does the 50-token restriction change?

Document length and the denominator (2/2)

`head(..., 3)` shows the top three rows; `cols` selects columns.

Sorted by	doc_id	broad	tokens	broad_per100
Count	speech_2792	20	1,349	1.483
Count	speech_1673	19	2,655	0.716
Count	speech_0743	18	1,599	1.126
Rate	speech_4435	4	69	5.797
Rate	speech_1551	4	78	5.128
Rate	speech_4917	3	59	5.085

Rate = 100 × broad / tokens. `broad` counts matches to `bill*`.

20 / 1,349 → 1.48 per 100; 4 / 69 → 5.80 per 100.

More matches does not necessarily mean a higher density.

Actual C09 output, rounded to three decimals; discovery texts with at least 50 tokens. Row names repeat doc_id and are omitted. Broad bill-prefix matches do not establish household hardship.

Case: measuring evaluations in German politics

- Can word counts recover **political evaluations**?
- Adapt existing German sentiment dictionaries.
- Compare their scores with human judgments.
- Test speeches, manifestos and news.

JOURNAL OF INFORMATION TECHNOLOGY & POLITICS
2018, VOL. 15, NO. 4, 319–343
<https://doi.org/10.1080/19331681.2018.1485608>

 **Routledge**
Taylor & Francis Group

 OPEN ACCESS  Check for updates

Validating a sentiment dictionary for German political language—a workbench note

Christian Rauh

WZB Berlin Social Science Center, Department Global Governance

From political sentences to a validated score

Step	What the study does
Dictionary	Combine SentiWS and GermanPolarityClues; manually review terms
Context	Add direct two-word negation rules
Score	Positive minus negative matches, divided by retained terms
Validation	Three people code 1,500 sampled Bundestag sentences

Compare the original dictionaries and the augmented version with human judgments.

Case: sentiment and legislative agreement

- Does opposition language reflect **cross-party agreement**?
- Translate Lexicoder sentiment dictionaries.
- Validate scores against human judgments.
- Relate debate tone to unanimous bill passage.



SVEN-OLIVER PROKSCH
University of Cologne
WILL LOWE
Princeton University
JENS WÄCKERLE
University of Cologne
STUART SOROKA
University of Michigan

*Multilingual Sentiment Analysis: A
New Approach to Measuring Conflict
in Legislative Speeches*

From speeches to a test of cross-party agreement

Question: Is more positive opposition language associated with unanimous passage?

1. Collect about **26,000 speeches on 871 German government bills**, 2005-2017.
2. Match positive and negative words with translated Lexicoder dictionaries.
3. Aggregate counts by party and bill: $\log \left(\frac{P + 0.5}{N + 0.5} \right)$.
4. Use bill-level sentiment in logistic regressions of unanimous passage.

Human-coded European Parliament texts provide a separate measurement check.

Two simple sentiment summaries

Let **P** = positive matches, **N** = negative matches, **L** = original token count.

Summary	Definition	Interpretation
Net tone	$100 \times (P - N) / L$	Net matches per 100 tokens
Match coverage	$100 \times (P + N) / L$	Matched sentiment tokens per 100 tokens

Scoring means turning matches into a number. With $P = 3$, $N = 1$ and $L = 100$: net tone = **2**, coverage = **4%**. These differ from Proksch et al.’s log-ratio score.

Zero is not always neutral (1/2)

`check_texts` contains five teaching examples, created in `workflow.R` and loaded by `C01`.

Name	What it checks
<code>good</code>	A positive match
<code>negated</code>	Whether negation is handled
<code>unmatched</code>	No dictionary matches
<code>balanced</code>	Positive and negative matches cancel
<code>empty</code>	No words; denominator is zero

```
# C10: Zero, balance and missing matches
check_texts
check_scores[, c("doc_id", "positive", "negative",
                 "net_per100", "coverage_per100", "balance")]
```

Compare: Why do `unmatched`, `balanced`, and `empty` differ?

Zero is not always neutral (2/2)

P = positive matches; N = negative matches; L = total word tokens. **Net** = $100 \times (P - N) / L$; **Coverage** = $100 \times (P + N) / L$; **Balance** = $(P - N) / (P + N)$.

Actual C10 output (rounded)

doc_id	Net	Coverage	Balance
good	25	25.00	1
negated	20	20.00	1
unmatched	0	0.00	NA
balanced	0	28.57	0
empty	NA	NA	NA

Net = `net_per100`; Coverage = `coverage_per100`;
Balance = `balance`.

What each row means

- **good**: 1 positive match / 4 words → 25 per 100; all matches positive.
- **negated**: 1 positive match / 5 words → 20. The rule ignores `not`.
- **unmatched**: text exists, but P = N = 0. Balance has a zero denominator.
- **balanced**: P = N = 1 across 7 words. Net = 0; coverage = $100 \times 2 / 7 = 28.57\%$.
- **empty**: L = 0 and P + N = 0. All three scores are undefined (NA).

Zero can mean no matches or cancellation; neither proves neutrality.

Negation and dictionary matches (1/2)

Predict: Will keeping `not` make the second sentence negative?

```
# C11: A deliberate sentiment failure (run C01 first)
negation_tokens <- check_tokens[c("good", "negated")]
as.list(negation_tokens)
negation_dfm <- dfm(negation_tokens)
negation_counts <- dfm_lookup(negation_dfm, toy_sentiment)
as.matrix(negation_counts)
without_stopwords <- tokens_remove(negation_tokens, stopwords("en"))
as.list(without_stopwords)
```

Explain: Both sentences contain `good`. Does our word-count rule use `not` to change its category?

Negation and dictionary matches (2/2)

1. Dictionary matches

	positive	negative
good	1	0
negated	1	0

Both sentences match `good`. The rule does not use `not` to reverse the match.

Keep negation words, then add a rule that handles them.

2. After stopwords removal

```
$good  
[1] "policy" "good"  
  
$negated  
[1] "policy" "good"
```

`not` is deleted. The two token lists are now identical.

A fixed negation window has limits

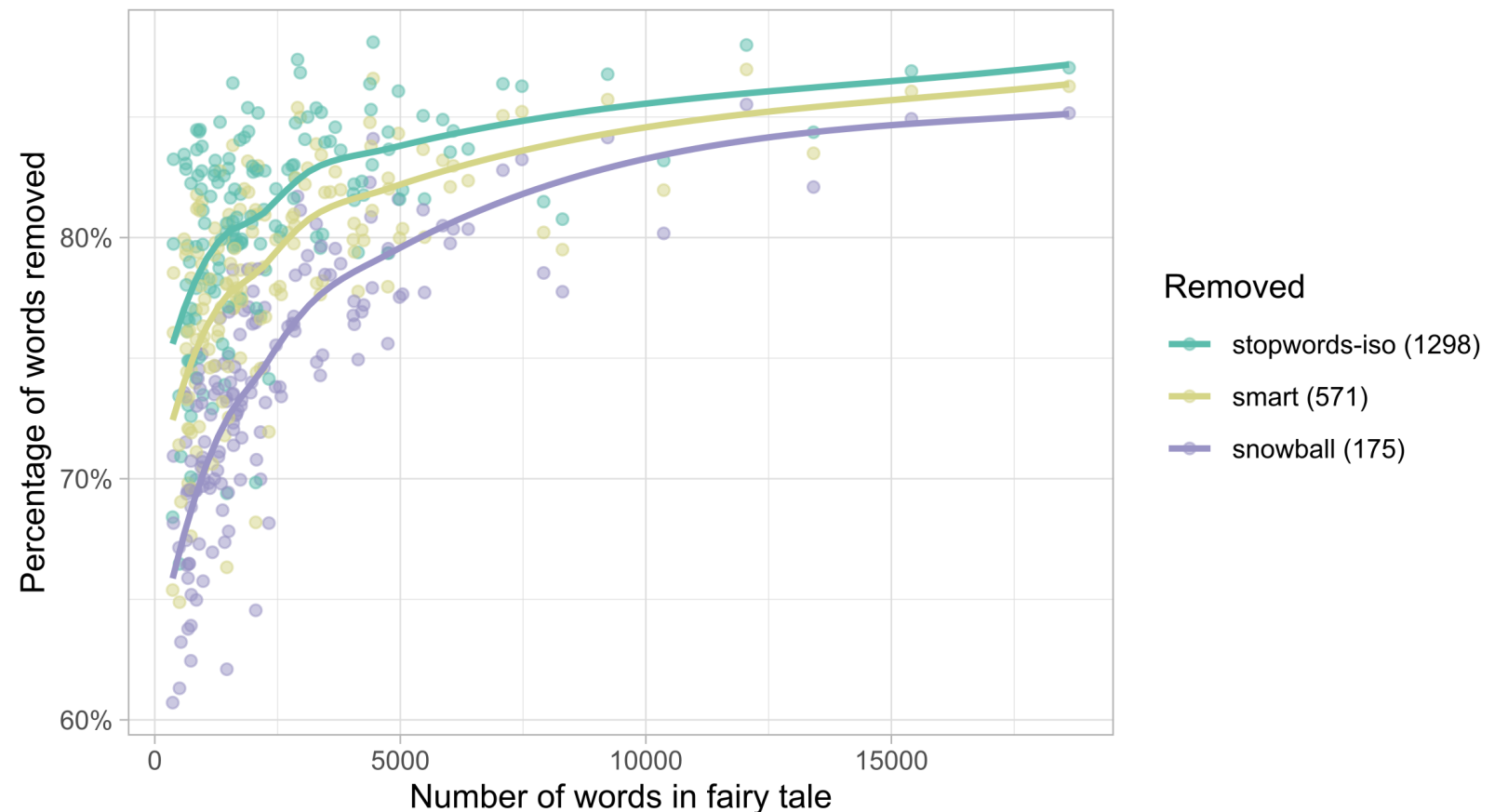
- “Not good” differs from “not only good but excellent.”
- “I reject the claim that this is a failure” contains negative words.
- Quoted criticism may not express the speaker’s own view.
- Long-distance negation can exceed a simple word window.

Rules can improve a baseline without resolving every sentence.

Preprocessing must fit the dictionary

- Stop-word lists remove **different shares** of a text.
- Keep negation when it carries meaning.
- Match dictionary entries to the chosen word forms.
- Check what each preprocessing choice discards.

Stop words take up a larger part of the text in longer fairy tales
Each vertical trio of points represents an H.C. Andersen fairy tale



Note: Each story has three points, one per stopword list. X = total words; Y = percentage of word occurrences removed. Colours identify lists; brackets show list sizes; curves show trends. Different lists remove different shares of the same text.

Case: locating parties from their manifestos

- Can manifesto language locate parties on a **left-right scale**?
- Eight British manifestos, 1983-1997.
- Wordfish estimates positions from word counts.
- Do preprocessing choices change the ordering?

Text Preprocessing For Unsupervised Learning: Why It Matters, When It Misleads, And What To Do About It*

Matthew J. Denny[†]

Arthur Spirling[‡]

[†]mdenny@psu.edu; 203 Pond Lab, Pennsylvania State University , University Park, PA 16802

[‡]arthur.spirling@nyu.edu; Office 405, 19 West 4th St., New York University, New York, NY 10012

12 orderings from the same manifestos

- Same eight manifestos and Wordfish model.
- **128** preprocessing specifications.
- **12 distinct orderings** of the documents.
- Black cells disagree with the authors' prior ordering.

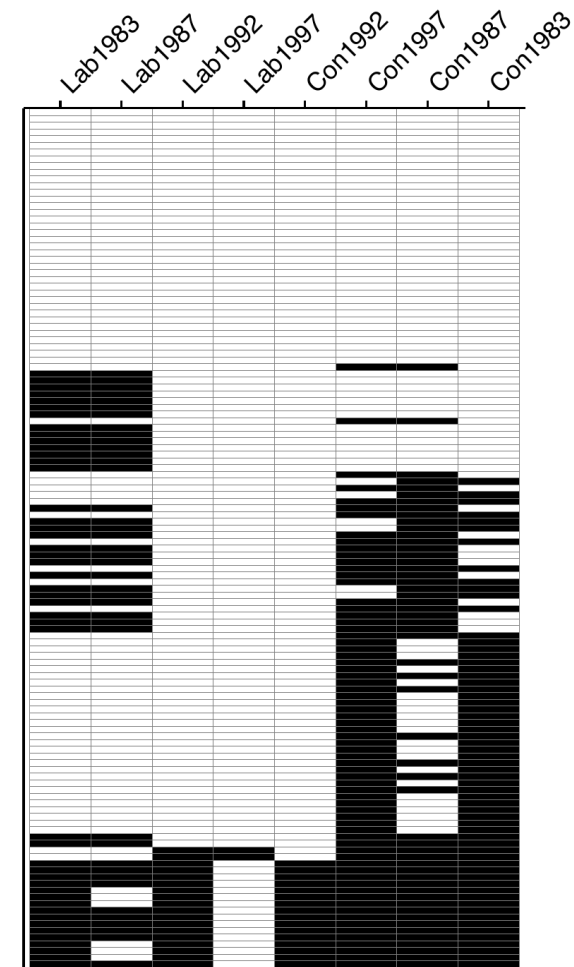


Figure 1: Wordfish results for the 128 different preprocessing possibilities. Each row of the plot represents a different specification. A white bar implies that the manifesto for that year is in the correct place as regards our priors. A black bar implies it was misplaced.

What the manifesto case can establish

Research claim	What this case shows
Vocabulary can place parties on a scale	Wordfish estimates relative positions from counts
The ordering survives reasonable choices	Check it: this example produces 12 orderings
The model explains why a party changed	That causal question needs a different design

For our dictionaries: do substantive comparisons survive reasonable scoring choices?

Your case: does a dictionary revision improve measurement?

Predict: Which version will flag more speeches?

```
# C12: Dictionary revisions (run C01 first)
policy_entries[, c("version", "category", "pattern")]
dev <- subset(policy_scores,
  doc_id %in% docnames(discovery_tokens))
c(v1 = sum(dev$v1 > 0), v2 = sum(dev$v2 > 0))
energy_context <- kwic(discovery_tokens, "energy", window = 5)
head(energy_context, 3)
```

Discuss: Scores changed. Do they agree better with human judgments?

Sentiment scores against human judgments

Return to Rauh: do dictionary scores recover human-rated evaluations?

- 1,500 Bundestag sentences, three coders.
- Compare three dictionaries across human-rated categories.
- Positive categories separate more clearly.
- Intervals show uncertainty in group means, not classification accuracy.

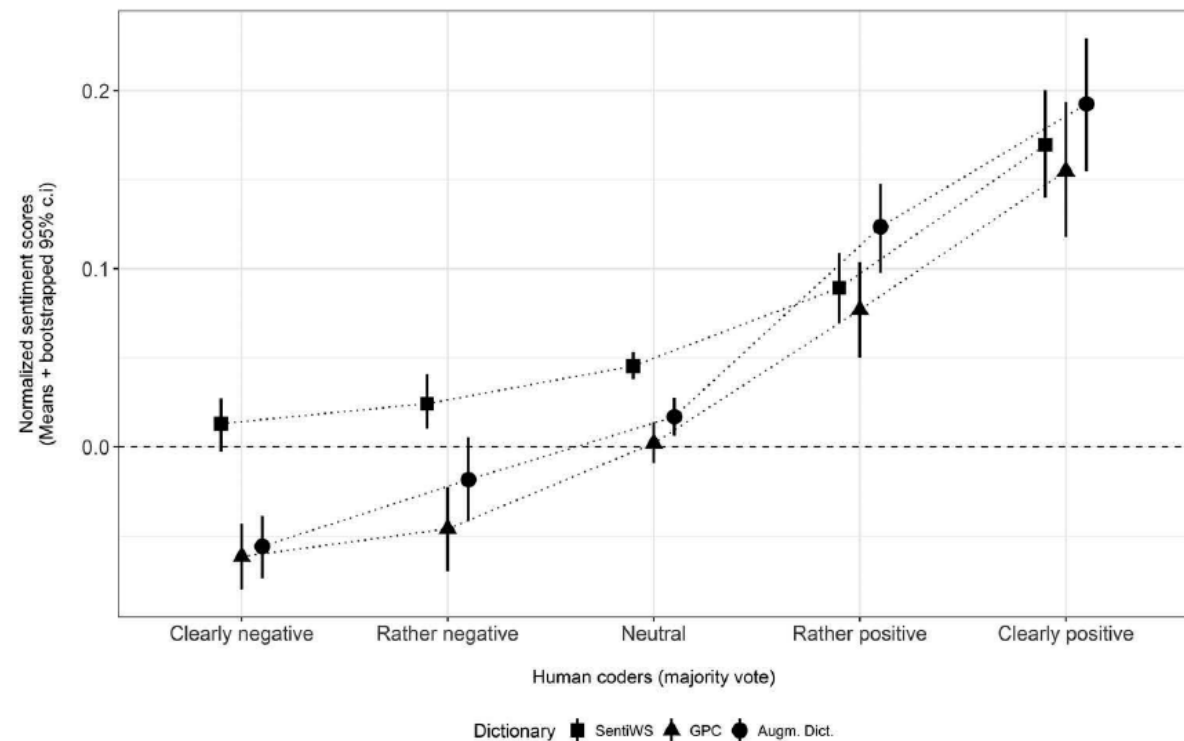


Figure 2. Human judgment against mean sentiment scores in 1,500 random Bundestag sentences.

Positive evaluations are easier to recover

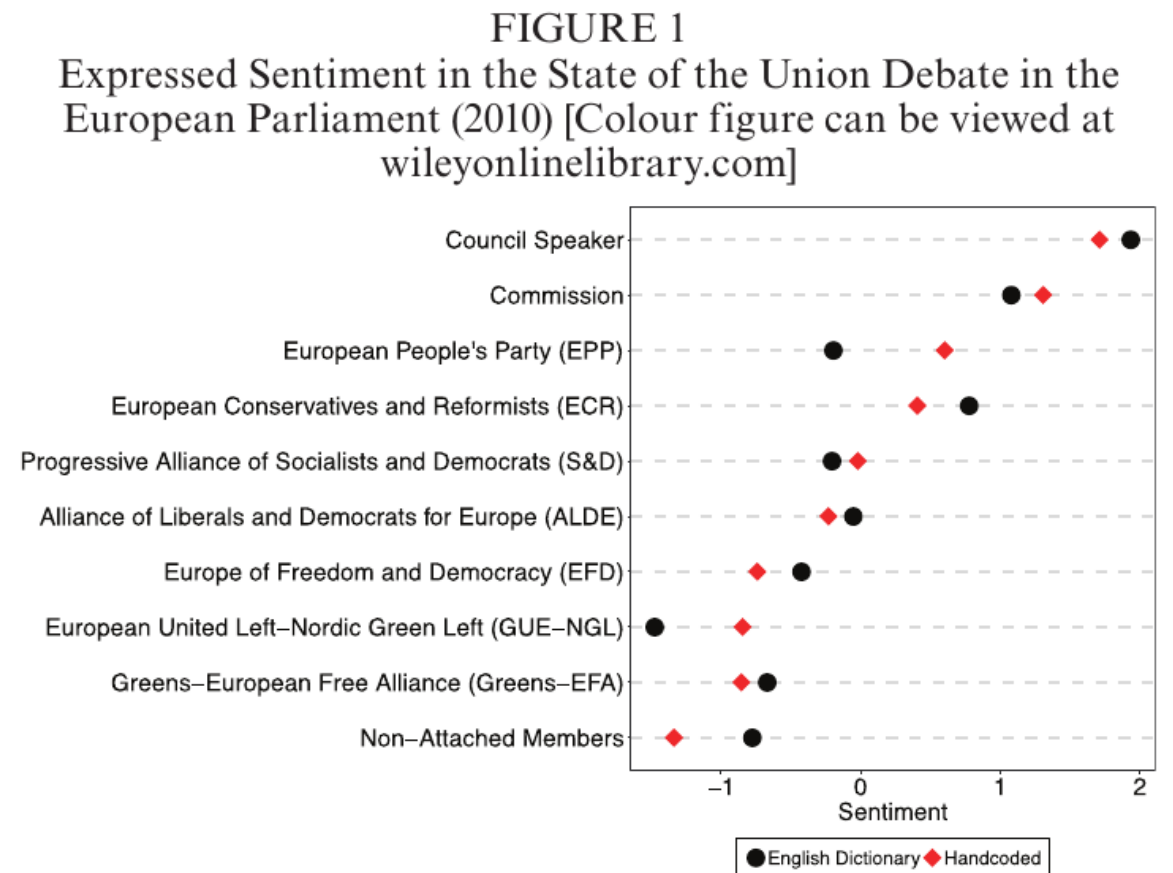
- **Result:** positive evaluations separate more clearly than negative ones.
- **Use:** a tested indicator for comparing evaluative language in political texts.
- **Limit:** scores cluster near zero and do not have self-evident numerical units.

The study validates a measure. It does not explain why politicians become more positive.

Human and automated scores by speaker group

Return to Proksch et al.: compare human and automated group-level tone.

- 300 paragraphs, two human ratings per paragraph.
- Compare standardized scores by speaker group.
- Reported group-level correlation: **about 0.9**.
- This is not 90% classification accuracy.



Note. The plot shows the expressed sentiment on the automated approach (English Lexicoder dictionary) and on hand-coded estimates for the relevant speaker groups (Council, Commission, and EP political groups). Because the two measures are on different scales (log ratio of counts and five-point Likert scale, respectively), we display z-transformed scores.

Validation depends on the level of the claim

Claim	Evidence needed
This sentence is negative	Sentence-level agreement
This group is relatively more positive	Group-level measurement validity
Sentiment changed over time	Stable measurement across periods
Countries differ in conflict	Comparable measures and institutional context

One successful comparison does not validate every use.

Translation is a measurement decision

- A translated term can have a different political use.
- Translation may produce several possible equivalents.
- Morphology and tokenization affect matches.
- Validate languages separately and examine difficult cases.

Shared category names do not guarantee comparable numerical scales.

Does opposition tone reflect agreement?

- Outcome: **unanimous bill passage.**
- **814 bills** in the regressions.
- Positive coefficients in all three models; only model (3) is significant.
- Association, not a causal effect of tone.

TABLE 1
Explaining Unanimous Voting on Government Bills in the German Bundestag, 2005–17

	Dependent Variable		
	Unanimity in Last Reading		
	(1)	(2)	(3)
Opposition Sentiment	0.521 (0.450)	0.763 (0.528)	1.155** (0.540)

Proksch et al. (2019), Table 1 excerpt, p. 120. Other rows omitted; ** p < .05. Standard errors in parentheses.

Sentiment, stance, and conflict

Concept	What the evidence must establish
Sentiment	Evaluative language in a defined unit
Stance	Support or opposition toward a specified target
Conflict	A substantive relationship between actors or positions

Negative words can describe a social problem without attacking another party.

A human benchmark

- Write a codebook with positive, negative, and ambiguous examples.
- Keep coders unaware of dictionary predictions when practical.
- Record disagreement and the adjudication procedure.
- Match the coding unit to the intended use of the score.

Human labels are evidence to scrutinize, not unquestionable truth.

Which texts should we validate?

- Include matched **and unmatched** passages.
- Cover important dates, groups, and document lengths.
- Use an untouched sample for final performance estimates.
- Inspect rare or ambiguous cases separately for diagnosis.

A collection of easy examples cannot establish corpus-wide performance.

Inter-coder reliability: do humans agree?

- Code the same texts **independently** before discussing differences.
- Rauh: **3 coders**, 1,500 sentences.
- **59.1%** full agreement; ordinal alpha **.675**.
- Agreement does not establish **validity**.

Table A1. Inter-coder Agreement

Coders	Percentage agreement	Krippendorff α (ord. data)
Human coders (3)	59.1	.675
Sentiment dictionaries (3)	77.2	.599
Humans and dictionaries (6)	24.4	.404

Note: A total of 1,500 randomly sampled sentences from Bundestag speeches coded as ‘positive,’ ‘neutral,’ or ‘negative.’

Agreement in R: two human coders

Extension: ten illustrative labels, 1 = energy-policy relevant, 0 = not relevant.

```
# C13: Human agreement, not model accuracy
# install.packages("irr") # run once
library(irr)
ratings <- data.frame(
  coder_A = c(1, 1, 1, 1, 1, 0, 0, 0, 0, 0),
  coder_B = c(1, 1, 1, 1, 0, 1, 0, 0, 0, 0))
table(ratings$coder_A, ratings$coder_B)
mean(ratings$coder_A == ratings$coder_B)
agreement <- kappa2(ratings, weight = "unweighted")
agreement$value
```

Discuss: What does kappa (.60) tell us beyond raw agreement (80%)?

A confusion matrix for issue detection

20 illustrative passages	Human: relevant	Human: not relevant
Dictionary: match	6 true positives	2 false positives
Dictionary: no match	4 false negatives	8 true negatives

The unit is a **passage**, not a dictionary word.

Illustrative counts for classroom practice; not empirical results from the lab corpus.

Precision and recall answer different questions (1/2)

Before running the code: calculate precision and recall from the preceding table.

```
# C14: Checking validation arithmetic (run C01 first)
predicted <- validation$predicted
human <- validation$human
TP <- sum(predicted == 1 & human == 1)
FP <- sum(predicted == 1 & human == 0)
FN <- sum(predicted == 0 & human == 1)
precision <- TP / (TP + FP)
recall <- TP / (TP + FN)
F1 <- 2 * TP / (2 * TP + FP + FN)
c(precision = precision, recall = recall, F1 = F1)
```

Discuss: Could we estimate recall by reading only dictionary matches?

Precision and recall answer different questions (2/2)

Code: `c(precision = precision, recall = recall, F1 = F1)`

```
precision    recall          F1
0.7500000  0.6000000  0.6666667
```

- **Precision = 0.75:** Of 8 passages the dictionary flags, **6 are relevant** (6 / 8).
- **Recall = 0.60:** Of 10 relevant passages, the dictionary **finds 6** (6 / 10).
- **F1 $\approx$ 0.667:** Combines precision and recall: $2 \times 6 / (2 \times 6 + 2 + 4)$.

Missed cases matter: reading only matched passages cannot tell us recall.

Evaluating continuous sentiment scores

- Plot scores against human ratings.
- Inspect group means **and** individual disagreements.
- Check unmatched texts, compressed ranges, and extreme values.
- Compare performance across groups and genres.

Two scores can rise together but still disagree: scores of 1, 2, 3 and 11, 12, 13 have perfect correlation.

Dictionary sensitivity (1/2)

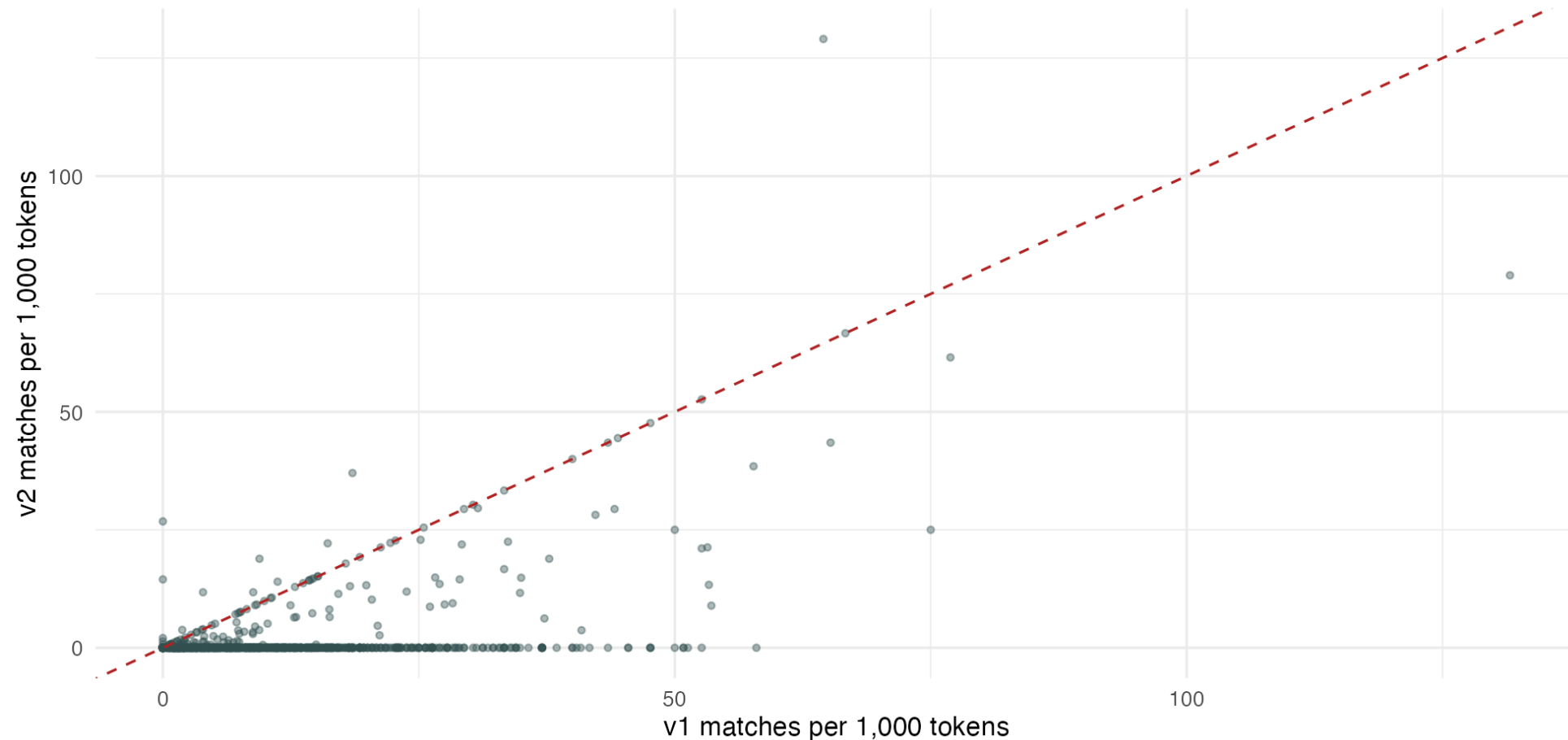
What is `dev`? The table created in **C12** for inspecting dictionary rules: **one row = one speech**, with 4,950 speeches.

`v1_per1000` and `v2_per1000` are its columns: matches per 1,000 words under each dictionary version.

```
# C15: Visualizing dictionary sensitivity (run C12 first)
# install.packages("ggplot2") # run once
library(ggplot2)
ggplot(dev, aes(x = v1_per1000, y = v2_per1000)) +
  geom_point(size = 1, alpha = 0.4, colour = "#314f4f") +
  geom_abline(intercept = 0, slope = 1,
              colour = "firebrick", linetype = "dashed") +
  labs(x = "v1 matches per 1,000 tokens",
       y = "v2 matches per 1,000 tokens") +
  theme_minimal()
```

Discuss: What can this plot show without human labels?

Dictionary sensitivity (2/2)



One point = one speech. On the dashed line: unchanged; below: fewer matches in v2; above: more matches in v2. Overlapping points can hide many speeches.

Changed scores show sensitivity, not which dictionary is more valid.

What a methods section should report

- Corpus boundaries, source, and unit of analysis.
- Dictionary origin, version, language, and modifications.
- Matching, preprocessing, denominator, and zero rules.
- Human validation design, results, and remaining errors.
- Code and access information sufficient to audit the workflow.

Embeddings and LLMs can propose candidates

Seed concept → suggested terms → corpus examples → researcher review

- Embeddings can suggest related expressions.
- LLMs can propose variants or organize candidate lists.
- Antonyms and irrelevant associations can also appear.
- Keep an independent human benchmark.

These tools assist dictionary development; they do not certify validity.

Extending the workflow to Chinese texts

Choice	Example issue
Segmentation	Does a relevant expression remain intact?
Script normalization	Are variants merged appropriately?
Entity aliases	Are abbreviations linked to the correct institution?
Negation and context	Does the phrase express support, criticism, or quotation?

Validate in the language and political setting of the project.

Applying the ideas to a research project

Choose one possible corpus and complete these sentences:

- I want to measure ..., rather than merely count
- My unit of analysis is
- A plausible false positive is
- A plausible false negative is
- I would validate the measure using

LIWC: what does it measure?

Application: use the same construction, scoring and validation principles.

LIWC = Linguistic Inquiry and Word Count.

- Matches language to predefined dictionary categories.
- Categories include pronouns, emotion and social processes.
- Most category scores are percentages of a text's words.

Example: 4 category matches in 100 words = 4%.

A category percentage measures word use; interpretation needs validation.

LIWC and quanteda: separate roles

Resource	Role	Access
LIWC application	Runs built-in or custom dictionaries	Licensed software
Official LIWC dictionaries	Proprietary word-category resources	Subject to license
quanteda + our dictionary	R matching with our own rules	No LIWC purchase

Importing a file does not grant a license to use its dictionary.

Campaign statements: vague or concrete?

- Eichorst & Lin (2019)
- When do parties make concrete campaign statements?
- Texts: English-language party programs.
- Compare government status and economic performance.

Resist to Commit: Concrete Campaign Statements and the Need to Clarify a Partisan Reputation

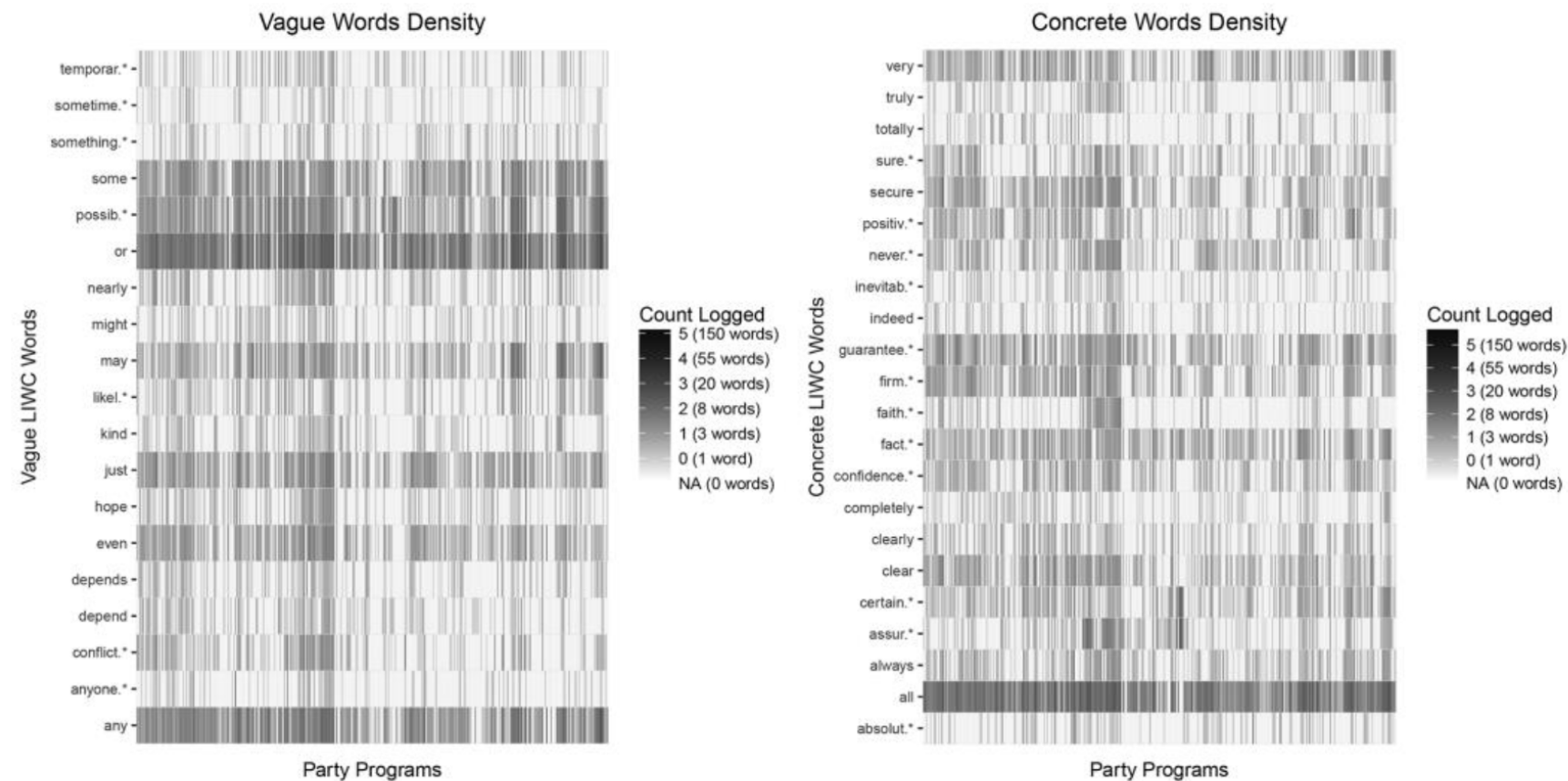
Jason Eichorst, University of Mannheim
Nick C. N. Lin, Academia Sinica

Democratic accountability relies on citizens to anticipate future governing behavior. We explore the strategic incentives for parties to shape voter expectations by generating vague or concrete campaign statements. Using an English-language dictionary, we scale electoral statements from all industrialized English-speaking nations to develop a measure of concreteness. Concrete statements can create electoral risks from unfulfilled expectations. Yet, political parties have incentives to use concrete statements to clarify reputation uncertainty associated with unclear informational cues. Political context shapes these incentives. Incumbent parties tend to dictate concrete statements to balance attributed responsibility for government outcomes and signal that they are competent managers. Strong government performance, however, reduces the incentive for incumbents to be concrete, as favorable outcomes reveal competent management. Opposition parties are unconstrained from these demands. The research reveals how political parties actively manage and balance the information that voters use in order to adjust the informative value of party reputation.

Party responsibility to the public, enforced in elections, implies that there be more than one party, for the public can hold a party responsible only if it has a choice. Again, unless the parties identify themselves with programs, the public is unable to make an intelligent choice between them.

—American Political Science Association (1950, 22)

Reading the word-density figure



Each column is a **party program**; each row is a **dictionary term**. Darker cells indicate larger logged counts; white indicates zero.

A two-sentence dictionary example (1/3)

Copy the whole block into R. Requires `quanteda`.

```
library(quanteda)
text <- c("We may improve public services.",
         "We guarantee better public services.")
dict <- dictionary(list(vague = "may", concrete = "guarantee"))
toks <- tokens(text, remove_punct = TRUE)
counts <- dfm(toks)
hits <- dfm_lookup(counts, dictionary = dict)
as.matrix(hits)
```

A two-sentence dictionary example (2/3)

docs	features	
	vague	concrete
text1	1	0
text2	0	1

- `text1`: **may** matches the vague category once.
- `text2`: **guarantee** matches the concrete category once.
- The values count words matched by our rules.

Try: Add `not` before `guarantee`. Will the count change?

A two-sentence dictionary example (3/3)

Teaching score = $100 \times (C - V) / L$

C = concrete matches; V = vague matches; L = all word tokens.

This is **concrete percentage – vague percentage**, in percentage points.

```
m <- as.matrix(hits)
100 * (m[, "concrete"] - m[, "vague"]) / ntoken(toks)
# text1 text2
#    -20    20
```

text1: $100 \times (0 - 1) / 5 = -20$; text2: $100 \times (1 - 0) / 5 = 20$.

Adapted from Eichorst & Lin (2019): the paper excludes stopwords from the denominator and rescales the difference to 0–1. Our example does neither.

LIWC in sport and exercise

- Lin (2023): a Chinese-language review.
- Explains dictionary categories and word-use rates.
- Reviews research on injury, self-efficacy and sport communication.
- **Discussion:** What would language reveal about a team?

● Journals ● OpenAccess

字詞分析工具（LIWC）的理論基礎及其在體育領域的應用

Linguistic inquiry and word count in sport and exercise

林瑋芳(Wei-Fang Lin)

《體育學報》 Volume 56 Issue 1 (2023/03) Pp. 1-16

[https://doi.org/10.6222/pej.202303_56\(1\).0001](https://doi.org/10.6222/pej.202303_56(1).0001)

Cite

Share

Save

Print Biblio.

Download

Chinese LIWC: a validation case

- **Question:** Do categories distinguish emotional contexts?
- **Data:** 50 Hate + 50 Sad posts from PTT.
- **Anger words:** 2.00% vs. 0.88%; Cohen’s $d = 0.61$.
- Group differences are **not** individual diagnoses.

表 3 研究三之各變項在 Hate 版與 Sad 版之描述統計與檢定指標

	Hate 版		Sad 版		t 檢定值	效果量 Cohen’s d
	平均數	標準差	平均數	標準差		
生氣詞	2.00	2.38	0.88	1.01	3.04**	0.61
髒話	1.45	2.24	0.13	0.32	4.14***	0.83
網路用語	2.39	2.38	1.18	1.07	3.28**	0.66
驚嘆號	0.67	1.52	0.11	0.35	2.57*	0.51
權力詞	3.18	1.99	2.69	1.73	1.32	0.26
認知複雜度	3.36	1.22	4.07	1.13	−3.02**	0.61
第一人稱單數代名詞	3.02	2.82	4.34	2.43	−2.51*	0.50
焦慮詞	0.40	0.65	0.89	1.01	−2.85**	0.56
悲傷詞	0.57	0.98	1.44	1.21	−3.92***	0.78
認知歷程詞	16.81	5.64	20.62	5.70	−3.36***	0.67

* $p < .05$; ** $p < .01$; *** $p < .001$

Our dictionary, applied with quanteda

Import our **ten-word teaching dictionary**, not an official LIWC dictionary.

```
# C16: Custom dictionary (run C01; stay in W4/classroom)
library(quanteda)
own <- dictionary(file = "Dictionaries/teaching_affect.dic", format = "LIWC")
mini <- tokens(c("Good policy.", "A bad crisis."), remove_punct = TRUE)
matched <- tokens_lookup(mini, own, capkeys = FALSE)
hits <- dfm(matched)
counts <- as.matrix(hits)
counts
word_count <- ntoken(mini)
100 * counts / word_count
```

Discuss: Why divide by all word tokens, not just dictionary matches?

Custom policy dictionary: inspect the errors

Optional challenge: Add one justified energy-policy expression.

```
# 002: Your own dictionary (run C01 first)
my_dict <- dictionary(list(energy = c("energy", "gas bills")))
matched <- tokens_lookup(discovery_tokens, my_dict)
my_dfm <- dfm(matched)
my_counts <- as.numeric(my_dfm[, "energy"])
sum(my_counts > 0)
my_context <- kwic(discovery_tokens, "energy", window = 6)
head(my_context, 3)
```

In pairs: Show one supporting passage and one possible false positive.

Validation: freeze, test and interpret

- **Develop:** use 12 teaching sentences to see what v1 and v2 get wrong.
- **Freeze:** keep the supplied v2 unchanged for the next 12 sentences.
- **Test:** compare its predictions with the supplied labels, including nonmatches.
- **Result:** precision **0.80**, recall **0.667**, F1 **0.727**.

Illustrative arithmetic, not estimated performance on Parliament.

Dictionaries: strengths and limitations

Would a dictionary be a good choice for your project?

- Identify **two strengths** and **two limitations**.
- How would you balance speed, transparency, and sensitivity to context?
- What validation evidence would support your choice?

Discuss in pairs for 2 minutes: choose a corpus and explain your decision.

Five sentences, three annotation methods

First, label the sentiment expressed in each sentence.

- 1. This policy is good.
- 2. This policy is bad.
- 3. This policy is not good.
- 4. Excellent, another tax increase.
- 5. The committee met on Tuesday.

Method	How it assigns labels
Dictionary lookup	Matches words to predefined categories.
Transformer classifier	Predicts labels using a trained model.
LLM prompting	Follows a prompt and coding instructions.

Which sentences need more than word matching?

Comparing time for the same five sentences

A hypothetical timing comparison, not a benchmark.

- 1. This policy is good.
- 2. This policy is bad.
- 3. This policy is not good.
- 4. Excellent, another tax increase.
- 5. The committee met on Tuesday.

Method	Time for all five
Dictionary lookup	0.02 seconds
Transformer classifier	0.30 seconds
LLM prompting	3.00 seconds

Would faster labels answer our research question? What validation is still needed?

Illustrative timings, not measurements. Run dictionary example. [Inference and batching](#).

Before W6: five texts and one difficult decision

W5 (October 9): no class. W6 (October 16): human coding and classification.

Prepare now

Label five texts with a short codebook

Explain one uncertain decision

Keep dictionary predictions separate

Develop in W6

Annotation quality and coder agreement

Refine definitions before training

Train and evaluate Naive Bayes / SVM

About 10 minutes of preparation; no model training or upload.

Take-home lab: custom dictionaries

45-60 minutes of core practice.

1. Read the teaching dictionary and calculate sentiment scores.
2. Change an energy-policy rule and inspect its matches.
3. Find false matches and missed cases in development examples.
4. Keep v2 fixed and calculate precision, recall and F1 on test examples.

Lab materials will be released separately.

Three assigned articles

Denny, M. J., and A. Spirling (2018). “Text Preprocessing for Unsupervised Learning: Why It Matters, When It Misleads, and What to Do about It.” *Political Analysis* 26(2): 168–189. [DOI](#)

Rauh, C. (2018). “Validating a Sentiment Dictionary for German Political Language: A Workbench Note.” *Journal of Information Technology & Politics* 15(4): 319–343. [DOI](#)

Proksch, S.-O., W. Lowe, J. Wäckerle, and S. Soroka (2019). “Multilingual Sentiment Analysis: A New Approach to Measuring Conflict in Legislative Speeches.” *Legislative Studies Quarterly* 44(1): 97–131. [DOI](#)

Readings and software references

Further readings

Grimmer, Roberts & Stewart (2022). *Text as Data*. Chapters 5 and 16.

Hvitfeldt & Silge. *Supervised Machine Learning for Text Analysis in R*. Online chapters on tokenization and stopwords.

Eichorst & Lin (2019). “Resist to Commit.” *The Journal of Politics* 81(1): 15–32.

Boyle et al. (2025). “Catalysts for progress? Mapping policy insights from energy research.” *Energy Research & Social Science* 121: 103955.

Software documentation

quanteda: [dictionaries](#), [KWIC](#) and text matching.

quanteda.textstats: frequency, collocations and keyness.

UDPipe / **TextRank**: annotation and [keyword extraction](#).

ggplot2: plotting dictionary scores.

Flair: named entity recognition (Python).

irr: agreement between coders.

LIWC: categories and word-use measures.